

INFO8006 Introduction to Artificial Intelligence

Exercise session 2: Games and adversarial search

RECAP

Games

A *game* is defined by

- the *states* $s \in \mathcal{S}$ and the *initial state* $s_1 \in \mathcal{S}$;
- the *actions* $\text{actions}(s) \subseteq \mathcal{A}$ available in state s ;
- the *transition model* $s' = \text{result}(s, a)$, the state reached by taking a in s ;
- a function $\text{player}(s) \in \{1, \dots, N\}$ that says whose turn it is in state s ;
- the *terminal states* $T \subseteq \mathcal{S}$, tested by $\text{terminal-test}(s)$, *i.e.* $s \in T$;
- a *utility* $\text{utility}(s, p)$, the payoff of player p when the game ends in $s \in T$.

The *game tree* is defined by s_1 , actions and result. A solution is a *strategy* $\pi_p : \mathcal{S} \rightarrow \mathcal{A}$, a move for every state the opponent can lead to.

Zero-sum games

A game is *zero-sum* if the utilities of the players sum to the same constant c in every terminal state, $\sum_{p=1}^N \text{utility}(s, p) = c$ for all $s \in T$. With two players, $\text{utility}(s, 2) = c - \text{utility}(s, 1)$: a single utility describes the game, which player 1 maximizes and player 2 minimizes. Below, the game is deterministic, turn-taking, two-player, zero-sum and of perfect information: MAX maximizes the utility, MIN minimizes it, and MAX moves first.

Minimax

The *minimax value* of s is the best payoff MAX can secure against an optimal MIN,

$$\text{minimax}(s) = \begin{cases} \text{utility}(s) & \text{if } s \in T \\ \max_a \text{minimax}(\text{result}(s, a)) & \text{if MAX plays} \\ \min_a \text{minimax}(\text{result}(s, a)) & \text{if MIN plays,} \end{cases}$$

over $a \in \text{actions}(s)$, and the optimal move is $\pi^*(s) = \arg \max_a \text{minimax}(\text{result}(s, a))$. Time $O(b^m)$, space $O(bm)$.

α - β pruning

α and β are the best values found so far for MAX and MIN on the path from the root: a MIN node stops as soon as its value $v \leq \alpha$, a MAX node as soon as $v \geq \beta$. The minimax values are unchanged and, with a perfect move ordering, $O(b^{m/2})$ nodes are examined. A *transposition table* stores $\text{minimax}(s)$ for every state s already evaluated, so that a state reached again by another sequence of moves is looked up rather than searched again.

H-Minimax

Under time constraints, an *evaluation function* $\text{eval}(s)$ replaces the utility and a *cutoff test* at depth d replaces the terminal test,

$$\text{h-minimax}(s, d) = \begin{cases} \text{eval}(s) & \text{if cutoff} \\ \max_a \text{h-minimax}(\text{result}(s, a), d + 1) & \text{if MAX plays} \\ \min_a \text{h-minimax}(\text{result}(s, a), d + 1) & \text{if MIN plays.} \end{cases}$$

The evaluation should order states as their minimax values do, and the cutoff apply to *quiescent* states only; what lies beyond the depth limit is hidden (*horizon effect*). An inaccurate evaluation can be compensated by a deeper search, as long as it becomes more accurate closer to the end of the game, but not the converse.

Multi-player games

Without the zero-sum assumption, or with $N > 2$ players, minimax backs up a vector of utilities $v(s) \in \mathbb{R}^N$, where the player to move maximizes its own component,

$$v(s) = \begin{cases} (\text{utility}(s, 1), \dots, \text{utility}(s, N)) & \text{if } s \in T \\ v(\text{result}(s, a^*)) & \text{otherwise,} \end{cases} \quad a^* = \arg \max_a v_{\text{player}(s)}(\text{result}(s, a)).$$

Expectiminimax

A *chance node* stands for a state-action pair (s, a) , from which the next state s' follows with probability $P(s' \mid s, a)$. With $v(s) = \text{expectiminimax}(s)$,

$$v(s) = \begin{cases} \text{utility}(s) & \text{if } s \in T \\ \max_a q(s, a) & \text{if MAX plays} \\ \min_a q(s, a) & \text{if MIN plays,} \end{cases} \quad q(s, a) = \sum_{s'} P(s' \mid s, a) v(s'),$$

where $q(s, a)$ is the value of the chance node (s, a) , and $P(\text{result}(s, a) \mid s, a) = 1$ for a deterministic action. With chance nodes, an evaluation function must be a positive linear transformation of the expected utility: preserving the order of states is not enough.

Monte Carlo tree search

A *random playout* plays the game to its end with random moves; the proportion of playouts won evaluates a state. Each node n of the search tree keeps the wins $Q(n, p)$, over the playouts through n , of the player p who moved into n , and its visits $N(n)$. A round does *selection*, *expansion*, *simulation* and *backpropagation*; selection picks the child n' of n that maximizes

$$\frac{Q(n', p)}{N(n')} + c \sqrt{\frac{\log N(n)}{N(n')}},$$

where p is the player to move in n . The move played is the one leading to the most visited child of the root.

In session: exercises 1, 2, 4 (b)–(d).

Exercise 1 Tic-tac-toe (AIMA ch. 5)

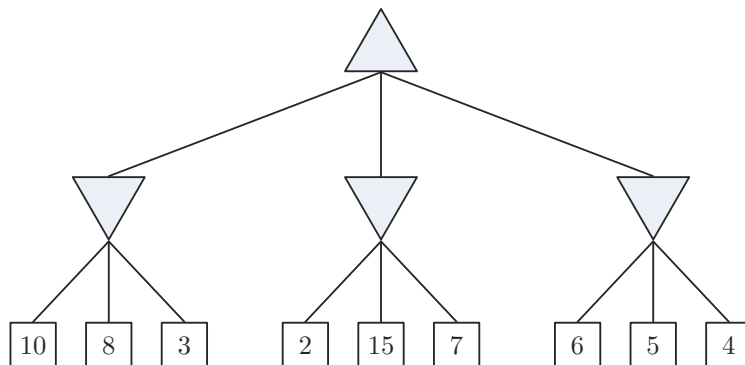
You are writing the engine of a program that plays Tic-Tac-Toe against human players. Tic-Tac-Toe is a game for two players, X and O, who take turns marking the cells of a 3×3 grid, X moving first. The player who succeeds in placing three of their marks in a straight line (horizontal, vertical or diagonal) wins the game. If neither of the players wins before the grid is full, it is a draw.

The program plays X, the MAX player, and its opponent O, the MIN player. We define X_n as the number of rows, columns or diagonals with exactly n X's and no O's. Similarly, O_n is the number of rows, columns, or diagonals with just n O's. A position s is terminal if $X_3(s) \geq 1$, $O_3(s) \geq 1$ or if the grid is full. The utility function assigns +1, -1 or 0 to such position, respectively. For non-terminal positions, we use an evaluation function defined as $\text{eval}(s) = 3X_2(s) + X_1(s) - 3O_2(s) - O_1(s)$.

- (a) Define the search problem associated with the Tic-Tac-Toe game.
- (b) Approximately how many possible game states of Tic-Tac-Toe are there?
- (c) Show the whole game tree starting from an empty grid down to depth 2 (one X and one O on the board), taking symmetry into account.
- (d) Annotate your tree with the evaluations of all the positions at depth 2.
- (e) Using the H-Minimax algorithm, annotate your tree with the backed-up values for the positions at depths 1 and 0, and use those values to choose the first move of the program.
- (f) Mark the nodes that α - β pruning would not evaluate if the nodes were generated in the optimal order.
- (g) Is this evaluation function a good heuristic? If not, provide one or more states s for which $\text{eval}(s)$ is misleading.

Exercise 2 Defending a network (CS188, Fall 2019)

A security program protects a computer network. It first chooses one of three configurations of the network; an attacker program, which observes the configuration, then chooses one of three attacks. The leaves of the tree below score the damage prevented: the defender maximizes it and the attacker minimizes it, so the game is zero-sum. The triangle that points up, at the root, is the choice of the defender (MAX); the triangles that point down are the choices of the attacker (MIN).



- (a) Assuming both programs act optimally, fill in the minimax value of each node. Which configuration does the defender choose?

- (b) Which nodes can be pruned from the game tree above through α - β pruning? If no nodes can be pruned, explain why not. Assume the search goes from left to right; when choosing which child to visit first, choose the left-most unvisited child.
- (c) Suppose now that there is no attacker, but random failures: once the configuration is chosen, one of its three outcomes happens uniformly at random. The MIN nodes become chance nodes. Fill in the expectiminimax value of each node. Which configuration does the defender choose, and why does it differ from (a)?

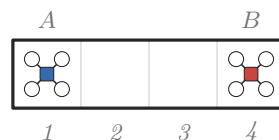
Exercise 3 21 misère game (Exam, January 2019)

A game is *solved* when the outcome of perfect play from its initial state is known, and a program that plays perfectly never loses; checkers was solved in 2007. We want to solve a small game. The game “21” is played with any number of players who take turns increasing a counter. The counter starts at 1 and each player in turn increases the counter by 1, 2, or 3; the player who says “21” or larger loses. Such a game, where the player who makes the last move loses, is a *misère* game. With two players, a state is a pair (v, p) of the value v of the counter and the player $p \in \{\text{MAX}, \text{MIN}\}$ to play, MAX playing first.

- (a) Consider the 2-player game of “5”, which has the same rules as “21” except that the player who says 5 or larger loses. Show the whole game tree.
- (b) Using the Minimax algorithm, annotate your tree with the backed-up values, and use those values to choose the optimal starting move. Who wins the game of “5” with perfect play?
- (c) Mark the nodes that α - β pruning would not evaluate if the nodes were generated in the optimal order.

Exercise 4 Drone race (AIMA ch. 5)

Two delivery drones, A and B , race in a narrow tunnel of four cells, shown below: A starts in cell 1 and must reach cell 4, and B starts in cell 4 and must reach cell 1. The drones move in turn, A first. At each turn, a drone must move to an adjacent free cell, in either direction. If the other drone occupies an adjacent cell, a drone may fly over it to the next free cell, if any. For example, if A is on 3 and B is on 2, then A may fly back to 1. The race ends when a drone reaches the opposite end of the tunnel. If A reaches cell 4 first, the value of the race to A is $+1$; if B reaches cell 1 first, it is -1 .



- (a) Define the search problem associated with this race.
- (b) Draw the complete game tree, using the following conventions:
- Put each terminal state in a square box and annotate it with its game value.
 - Put loop states (states that already appear on the path to the root) in double square boxes. Since their value is unclear, annotate them with a “?” symbol.
- (c) Explain why the standard minimax algorithm would fail on this game.
- (d) Annotate each node with its backed-up minimax value. Explain how you handled the “?” values and why.
- (e) This race can be generalized to a tunnel of n cells for any $n > 2$. Prove that A wins if n is even and loses if n is odd.

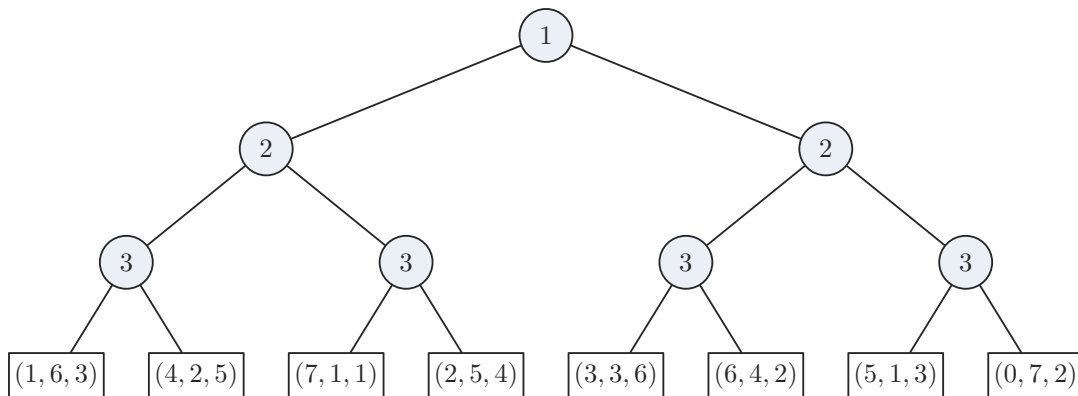
Exercise 5 Chess and transposition table (AIMA ch. 5)

Chess engines store the positions they have already evaluated in a transposition table, since many move orders lead to the same position. Suppose you write a chess engine that evaluates 16 million nodes per second (Deep Blue, which beat Garry Kasparov in 1997, evaluated about 200 million).

- (a) Decide on a compact representation of a game state for storage in a transposition table.
- (b) About how many entries can you fit in a 4 GB in-memory table?
- (c) Will that be enough for the three minutes of search allocated for one move?
- (d) How many table lookups can you do in the time it would take to do one evaluation? Suppose that you have a 3.2 GHz machine and that it takes 20 operations to do one lookup on the transposition table.

Exercise 6 Self-driving cars at an intersection

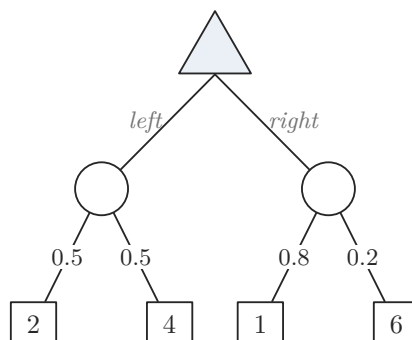
Three self-driving cars reach an intersection and decide in turn how to cross it, each between two maneuvers and after observing the choices of the cars before it: car 1 at the root of the tree below, car 2 at the second level and car 3 at the third. Each leaf gives the time saved by the three cars, (u_1, u_2, u_3) , and each car maximizes its own.



- (a) Back up the utilities to the root. Which maneuver does car 1 choose? Is the game zero-sum?
- (b) Car 1 now drives defensively: it assumes that the two other cars act against it and runs minimax on u_1 , as MAX, with cars 2 and 3 as MIN. Which maneuver does it choose, and what does it get if cars 2 and 3 in fact drive as in (a)?

Exercise 7 Evaluation functions in backgammon

A backgammon program chooses a move, after which the dice are rolled. Its evaluation function, learned from games against itself as in TD-Gammon, scores the positions reached, on the leaves of the tree below; the probabilities of the dice outcomes are on the edges.



- (a) Compute the expectiminimax value of each node. Which move does the program choose?
- (b) The developers rescale the scores x of the evaluation function, into $f(x) = 2x + 1$ or into $g(x) = 2^x$, which both keep the ranking of the positions. Which move does the program choose in each case?
- (c) Would g change the move if the chance nodes were MIN nodes? What does this say of the evaluation functions of H-Minimax, with and without chance nodes?

Exercise 8 Selection in Monte Carlo tree search

A Go program chooses its moves with Monte Carlo tree search, as AlphaGo does. During a selection step, the program, player p , is to move at a node n that has been visited $N(n) = 20$ times. Its three children are candidate moves, whose wins for p and visits are given below.

Child n'	n'_1	n'_2	n'_3
Wins $Q(n', p)$	8	3	1
Visits $N(n')$	12	6	2

- (a) Which child does UCB1 select with $c = 0$? With $c = 1$? Use the natural logarithm.
- (b) For which values of c does UCB1 select n'_1 ?

QUIZ

1. In Monte Carlo Tree Search (MCTS), in the formula

$$\frac{Q(n', p)}{N(n')} + c \sqrt{\frac{\log N(n)}{N(n')}},$$

which of the following is true?

- ☐ The first term encourages the exploitation of higher-reward nodes, while the second encourages the exploration of less-visited nodes.
 - ☐ The first term encourages the exploration of less-visited nodes, while the second term encourages the exploitation of higher-reward nodes.
 - ☐ The first term encourages the exploitation of highly-visited nodes, while the second term encourages the exploration of lesser-rewarding nodes.
 - ☐ The first term encourages the exploration of lesser-rewarding nodes, while the second term encourages the exploitation of highly-visited nodes.
2. Which of the following is true in adversarial search?
 - ☐ The horizon effect arises when the search is stuck in a cycle.
 - ☐ The horizon effect arises when the evaluation function is perfect.
 - ☐ The deeper in the tree the evaluation function is buried, the more the quality of the evaluation matters.
 - ☐ If not looked deep enough, bad moves may appear as good moves, because their consequences are hidden beyond the search horizon.
 3. A quiescent state is
 - ☐ A state in which the game will loop indefinitely.
 - ☐ A state in which an agent is stuck.
 - ☐ A state whose value is unlikely to exhibit wild swings in the near future.
 - ☐ A state in which the MAX agent is certain to win.

4. Which of the following is true of the minimax algorithm?

- ☐ It cannot fail more against a sub-optimal opponent than against an optimal one.
- ☐ It may be outperformed by a strategy that exploits a sub-optimal but predictable opponent.
- ☐ It does not ensure that MAX always wins.
- ☐ All of the above.